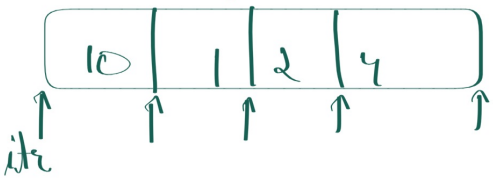
Day => 4

Iterable Interference =>

- -> Super interface of collection classes
- -> Simplifies Access to collections
- -> Part of java.lang Package

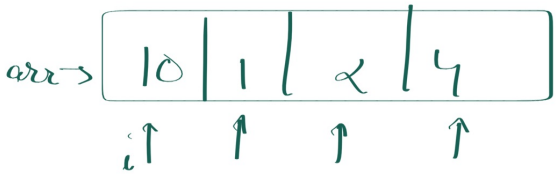
Iterator => not a index but a pointer

ArrayList



itr (iterator) is between the object

Normal array



arr[1] -> index i

pointing at the primitive data type

Note => [Any class which is implemented iterable interface can use for-each loop directly]

```
ArrayList<Integer> list = new ArrayList<>();
        list.add(1);
        list.add(2);
        list.add(20);
    for(int num: list){
            System.out.println(num
    }
```

Here ArrayList implements iterable interface

so it used for-each loop directly

As for-each loop internally uses → iterator

if this is true then

```
ArrayList<Integer> list = new ArrayList<>();
    list.add(1);
    list.add(2);
    list.add(20);
for(int num: list){
    if(num==1)
        list.remove(Integer.valueOf(1))
}
```

want to remove object 1

↓

This will give exception

But why?

As there is a variable (modCount) in the every iterable vectors and

when for-each loop internally

called Iterable class then in for-each loop mode count increase but the iterator mode count remain same and before printing every object it will check whether it has same expected mode count as of iterator. If not match it will return ConcurrentModificationException.

Iterator using while

```
Iterator<Integer> itr = list.iterator();
while(itr.hasNext()){
    System.out.println(itr.next());
}
```

Iterator using for

```
for(Iterator<Integer> itr = list.iterator(); itr.hasNext();){
    System.out.println(itr.next());
}
```

Every class which use for-each loop will implement the iterable interface

```java
class jennyLecture implements Iterable<String> {  2 usages
    private String[] data = {"j", "e", "n"};  1 usage
//    public String[] getData(){
//        return getData();
//    }
    // no need to use this as it is make iterable

    @Override
    public Iterator<String> iterator() {
        return Arrays.asList(data).iterator();// conversion of the Arrays.asList()-> into objects and then iterator is applied
    }

}
```

```java
//    so the class must implement the iterable interface
        jennyLecture jl=new jennyLecture();// in this as jennyCollection implements the iterable interface so we explicitly use for-each loop
        for(String s:jl){
            System.out.println(s);
        }

// -> in this we can also use the lembda expression and further it can be replaced by the method refrences
        List<Integer>l1=new ArrayList<>(List.of(1,2,3,4,5));
        l1.forEach( Integer n-> System.out.println(n));// in this action is performed
        // by method refrences
        l1.forEach(System.out::println);
        // println-> it is a method in the string class
        // System.out -> it is instance of class/ return object

//-> as we are using now multiply in each of the elements of list -> how we do that?
        List<Integer>l2=new ArrayList<>(List.of(1,2,3,4,5));
        l2.forEach(DAY04::add);
        // DAY04::add is a method reference

        l2.forEach( Integer a->DAY04.add(a));

        for(Integer a:l2){
            DAY04.add(a);
        }
// -> the above three statements are all equivalent

        ArrayList<String>s=new ArrayList<>(List.of("h","k","k"));
```